HDOC Day-53

****

Write a C program that reads the size and elements of a variable-length integer array, nums[]. Pass the array to a function named product_except_self() using its address. Use pointers to pass the array between the calling function and product_except_self(). The function product_except_self() must:

- Create an output array, answer[], of the same size as nums[].
- Store in answer[i] the product of all elements of nums[] except nums[i].
- Ensure that the product does not include nums[i].
- Display the elements of answer[] in their original order, separated by commas.
- Avoid using division to calculate the products.
- Use the fact that the product of every prefix and suffix fits within a 32-bit integer.

1. Problem Understanding

The program reads the size and elements of a variable-length integer array. The array is passed by address to product_except_self(). For every index, the function finds the product of all other elements without using division. A prefix product pass and a suffix product pass are used, so zeros and negative values are handled correctly. The resulting answer[] array is displayed in the original order, separated by commas.

2. Standard Test Cases

Test Case	Size (n)	Array Input	Expected Output
1	4	1 2 3 4	24, 12, 8, 6
2	5	-1 1 0 -3 3	0, 0, 9, 0, 0
3	3	2 3 4	12, 8, 6
4	4	0 2 3 4	24, 0, 0, 0
5	5	1 1 1 1 1	1, 1, 1, 1, 1

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length array nums[n].
4. Read n elements into nums[].
5. Pass nums and n to product_except_self() using the array address.
6. Stop.

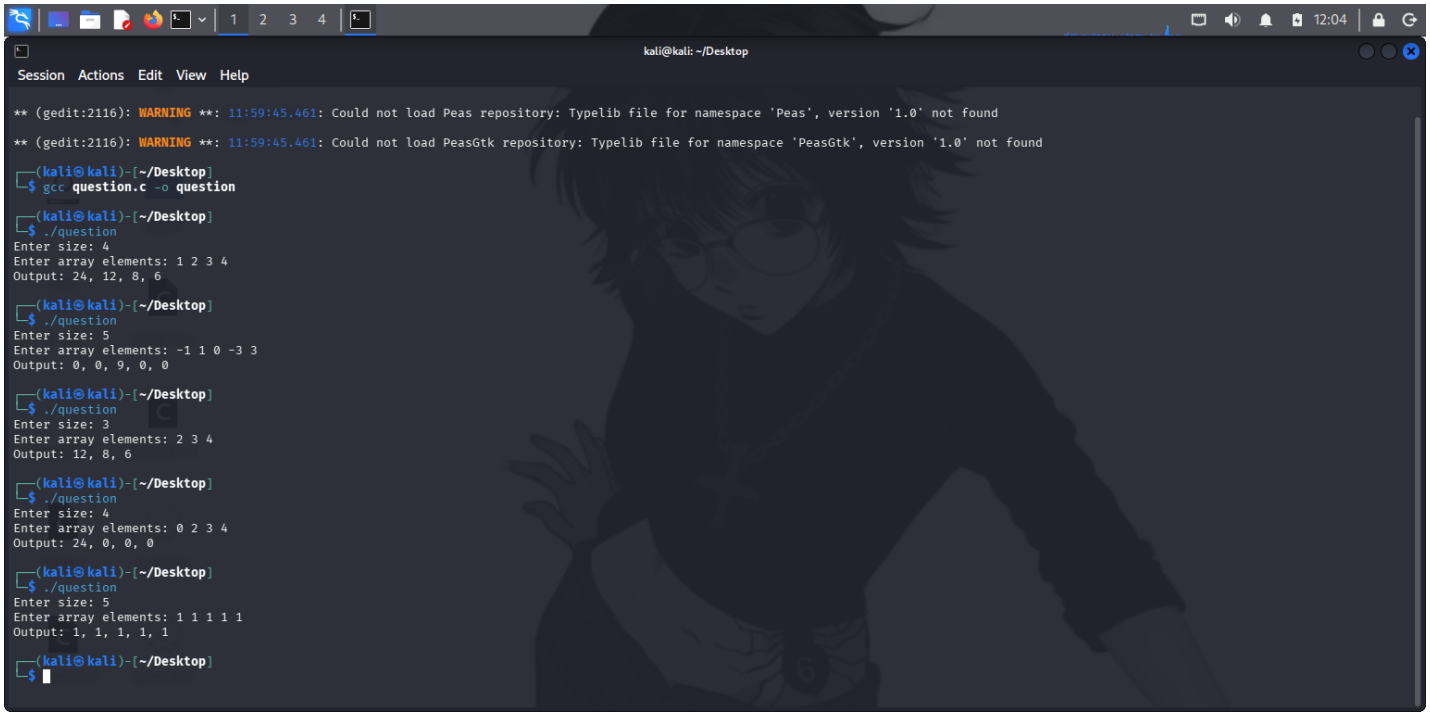
Function: product_except_self()

1. Create an output array answer[n].
2. Set prefix = 1.
3. Traverse from left to right: store prefix in answer[i], then multiply prefix by nums[i].
4. Set suffix = 1.
5. Traverse from right to left: multiply answer[i] by suffix, then multiply suffix by nums[i].
6. Display answer[] in order, separated by commas.
7. No division is used.

4. C Program

```
#include <stdio.h>
void product_except_self(int *nums, int n)
{
   int answer[n];
   int prefix = 1, suffix = 1;
   int i;
   for(i = 0; i < n; i++)
   {
      answer[i] = prefix;
      prefix = prefix * *(nums + i);
   }
   for(i = n - 1; i >= 0; i--)
   {
      answer[i] = answer[i] * suffix;
      suffix = suffix * *(nums + i);
   }
   for(i = 0; i < n; i++)
   {
      printf("%d", answer[i]);
      if(i < n - 1)
         printf(", ");
   }
   printf("\n");
}
int main()
{
   int n, i;
   printf("Enter size: ");
   scanf("%d", &n);
   int nums[n];
   printf("Enter array elements: ");
   for(i = 0; i < n; i++)
      scanf("%d", &nums[i]);
   printf("Output: ");
   product_except_self(nums, n);
   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	Actual Result	Status
1	4	1 2 3 4	24, 12, 8, 6	Pass
2	5	-1 1 0 -3 3	0, 0, 9, 0, 0	Pass
3	3	2 3 4	12, 8, 6	Pass
4	4	0 2 3 4	24, 0, 0, 0	Pass
5	5	1 1 1 1 1	1, 1, 1, 1, 1	Pass